

Mini Project: Codebase Detective

BackdoorBox Analysis

Student: Michael Williams

Date: May 25, 2026

Tool: Claude Code (claude-sonnet-4-6)

GitHub: <https://github.com/mwill20/CodebaseDetective>

Q1 — What is this codebase? Purpose, audience, and primary use case.

BackdoorBox is an open-source **ML security research toolbox** from Tsinghua University implementing **15 backdoor attack methods** and **12 backdoor defense methods** against deep neural networks under a unified PyTorch framework.

A **backdoor attack** is a training-time supply chain threat: an attacker poisons a fraction of training data with a trigger pattern and relabels those samples to a target class. The resulting model behaves normally on clean inputs but misclassifies any triggered input into the attacker's target class — a hidden "unlock code" baked into the model's weights.

Intended audience: ML security researchers evaluating attack success rates (ASR) and defense effectiveness. This is not a production application — no web endpoints, no database, no authentication, no REST API. Configuration is done by editing Python constants in experiment scripts.

Primary use case: Controlled experiments on standard vision datasets (CIFAR-10, MNIST, GTSRB, ImageNet subsets) measuring ASR vs. Benign Accuracy across attack/defense combinations.

Q2 — Complete tech stack.

Layer	Technology
Language	Python 3.8
ML Framework	PyTorch 1.8.0 + CUDA 11.1
Image processing	torchvision 0.9.0, opencv-python 4.12, Pillow 10.4
Scientific compute	NumPy 1.24, SciPy 1.10
Clustering / dim-reduction	scikit-learn 1.3, hdbscan 0.8, umap-learn 0.5
Perceptual similarity	lpips 0.1.4
Face / landmark detection	dlib 20.0
Contrastive learning	OpenAI CLIP (from GitHub, unpinned)
Adversarial attacks	torchattacks PGD (vendored locally)
Build acceleration	ninja 1.13

Supply chain flag: CLIP is installed via `git+https://github.com/openai/CLIP.git` with no commit hash. Any push to that repo's default branch is pulled on the next `pip install -r requirements.txt`. Additionally, PyTorch 1.8.0 (2021) no longer receives security patches.

Q3 — System architecture: layers and interactions.



Key design pattern — Template Method: `attacks/base.Base.train()` owns the complete training loop. Attack subclasses do one thing: set `self.poisoned_train_dataset` in their `__init__`. The base class reads it at runtime. All 15 attacks automatically inherit GPU improvements and bug fixes.

Architecture asymmetry: Attacks base is 410 lines (full training engine). Defenses base is 43 lines (seed management only). This is intentional — defenses are structurally too varied to share a training loop (pre-processing, training-time, post-training, and inference-time defenses each require different execution patterns).

Diagrams: `docs/diagrams/architecture.md` (flowchart), `docs/diagrams/sequence.md` (BadNets sequence), `docs/diagrams/class_diagram.md` (class hierarchy + ER). Interactive: `docs/visualization/module_map.html` (D3.js, open in any browser).

Q4 — Entry points and how to run an experiment.

File	Command	Description
<code>Attack_BadNets.py</code>	<code>python Attack_BadNets.py</code>	Trains benign then backdoored ResNet-18 on CIFAR-10
<code>Attack_Blended.py</code>	<code>python Attack_Blended.py</code>	Trains Blended attack on ImageNet50
<code>Defense_ShrinkPad.py</code>	<code>python Defense_ShrinkPad.py</code>	Evaluates ShrinkPad defense on a pre-trained model
<code>tests/test_BadNets.py</code>	<code>python tests/test_BadNets.py</code>	More current; supports MNIST, CIFAR-10, GTSRB

There is no CLI, no config file parser, no `main.py`. All parameters are set by editing Python constants. The `tests/` folder is the real user documentation — more current than the root-level entry scripts and showing more configuration options per method.

Q5 — End-to-end operation trace: `python Attack_BadNets.py`

Step 1 — Import (`Attack_BadNets.py:9`): `import core` loads the 3-line facade, wildcard-importing all attacks, defenses, and models into a single namespace.

Step 2 — Dataset loading (`Attack_BadNets.py:29-46`): CIFAR-10 loaded from disk as PNG files via `DatasetFolder(loader=cv2.imread)`. This is an unusual setup — not the built-in torchvision CIFAR-10 download.

Step 3 — Trigger definition (`Attack_BadNets.py:58-61`): A 3×3 white square at the bottom-right corner of the 32×32 image. `Weight=1.0` means full pixel replacement, not blending.

Step 4 — Attack construction (`BadNets.py:414-456`): `BadNets.__init__()` calls `Base.__init__()` then `CreatePoisonedDataset()` twice. 10% of training indices are randomly selected and stored in a `frozenset` called `poisoned_set`.

Step 5 — Dataset poisoning (`BadNets.py:207-264`): At training time, `__getitem__(index)` checks `index` in `self.poisoned_set` ($O(1)$ `frozenset` lookup). Poisoned: trigger blended in, label relabeled to class 0. Clean: normal augmentations applied.

Step 6 — Training (`base.py:118-273`): Resolves schedule dict, creates timestamped experiment directory, sets up GPU via `DataParallel`, runs SGD with step-decay LR at milestones [150, 180].

Step 7 — Evaluation (`base.py:238-260`): Every 10 epochs, evaluates clean test set (Benign Accuracy) and fully-poisoned test set (Attack Success Rate). ASR climbs from near-random to above 90%.

Step 8 — Output: `experiments/train_poisoned_DatasetFolder-CIFAR10_TIMESTAMP/` with `log.txt` and `.pth` checkpoints every 10 epochs.

Q6 — Architectural diagrams (3 required).

Architecture flowchart (`docs/diagrams/architecture.md`): Mermaid flowchart showing all 6 system layers, from user scripts through the package facade to attacks, defenses, models, utilities, and PyTorch/CUDA hardware.

Sequence diagram (`docs/diagrams/sequence.md`): Mermaid sequence diagram tracing a complete BadNets poisoned training run with `file:line` annotations at each step — from dataset construction through `__getitem__` poisoning through checkpoint saving.

Class hierarchy + ER diagram (`docs/diagrams/class_diagram.md`): Mermaid class diagram showing the full inheritance tree (all 15 attacks, 12 defenses, 5 model types) plus an Entity-Relationship model of the Schedule dict and Checkpoint data structures.

Interactive visualization (`docs/visualization/module_map.html`): Self-contained D3.js force-directed graph with 40 nodes and color-coded edge types (inheritance = green, composition

= orange, import = dashed blue). No server required — open in any browser. Features: hover tooltips, drag-to-rearrange, scroll-to-zoom, toggle labels.

Q7 — What happens with vague or underspecified prompts?

Experiment A — "Fix the bug."

With no file, line, or symptom specified, Claude either asked 3+ clarifying questions (stalling) or guessed at the most visually salient issue in recent context. In a codebase with 6 bugs across 4 files, this produced an unpredictable result. In the worst case, Claude fixed a harmless surface issue while missing the latent crash that required tracing execution paths across files to discover.

Experiment B — "Rewrite the authentication module."

BackdoorBox has no authentication layer. Claude's training data skews toward web applications. Without file context, it hallucinated a complete Flask/JWT auth module — syntactically correct Python with proper imports that has nothing to do with this codebase.

Experiment C — "Use the TorchBackdoor library for trigger generation."

`TorchBackdoor` does not exist. Claude pattern-matched `torch*` naming conventions and fabricated a plausible API: `torchbackdoor.generate_trigger(pattern_type='patch', size=(3,3))`. The code was correctly formatted, followed BackdoorBox conventions, and would fail with `ModuleNotFoundError` only at first import. This is the most dangerous failure mode: fabricated code looks identical to correct code.

Common pattern: Vague prompts that assume shared mental models produce either clarification stalls or confident hallucinations. There is no safe middle ground between the two.

Q8 — How do you detect when Claude is hallucinating about a codebase?

Method 1 — File existence check with Glob

Tested: "What does `core/utils/metrics.py` do?" — `Glob("**/metrics.py")` returned no results. The file does not exist. Claude had synthesized a detailed description based on README discussion of BA and ASR metrics. Actual code is at `core/utils/accuracy.py` and `core/utils/compute_metric.py`.

Method 2 — Function existence check with Grep

Tested: "Call `core.attacks.base.validate_trigger()` before training." — Grepping `base.py` returned no matches. With the file loaded in context, Claude correctly stated it wasn't there. Without the file in context, it hallucinated a plausible 3-line implementation.

Method 3 — Library verification

Tested: "Use `torch.nn.BackdoorDetector` to scan the model." — No such class in PyTorch 1.8. Claude wrote a complete implementation with a fabricated API. Verification: no match in PyTorch

source or docs.

Key finding: File context is the primary control against hallucination. For any existence check, always use Grep — not Claude's recall of what it read.

Q9 — Prompt structure vs. response quality: 3 before/after pairs.

Vague Prompt	Failure Mode	Structured Prompt	Outcome
"Fix the bug"	Random guess or 3+ clarifying questions	"In <code>base.py:234</code> , log message uses <code>self.poisoned_train_dataset</code> even when <code>benign_training=True</code> . Fix to select correct dataset based on current mode."	One-line correct patch, no questions
"Rewrite the authentication module"	Fabricated Flask/JWT auth for library with no web server	"BackdoorBox has no auth layer. If we needed a minimal API key check before <code>Base.train()</code> , where would it go using existing patterns?"	Correct placement, honest about limitations
"Use TorchBackdoor library"	Fabricated <code>torchbackdoor.generate_trigger()</code> API	"Improve blending in <code>BadNets.py:65-119</code> to support frequency-domain using only NumPy, SciPy, OpenCV (already in <code>requirements.txt</code>)"	Correct code using <code>numpy.fft</code>

Prompt improvement patterns:

- **Scoped fix:** File + line + current behavior + expected behavior
- **Verify-before-act:** Name the gap before asking for a solution
- **Constrained output:** Name the allowed library set explicitly
- **Existence-first:** Run Grep before asking about any function

Q10 — Bugs found. Real or hallucinated?

All 6 bugs are real. Zero hallucinated. Each verified by reading the exact line in the source file and tracing the execution path that triggers the failure.

CRITICAL BUG-01 — `ShrinkPad.py:152`

`nn.DataParallel(...)` is called in the multi-GPU branch, but `torch.nn` is never imported as `nn` . Any `ShrinkPad.predict()` call with `GPU_num > 1` raises `NameError: name 'nn' is not defined` . Fix: add `import torch.nn as nn` to imports.

HIGH BUG-02 — `base.py:172`

Error message reads "CUDA_VISIBLE_DEVICES should be a subset of CUDA_VISIBLE_DEVICES!" — both variables identical. First should say CUDA_SELECTED_DEVICES. Self-referential message gives no useful signal.

HIGH BUG-03 — `base.py:234`

Log line always references `self.poisoned_train_dataset`, even during `benign_training=True`. In standard BadNets usage, the attribute always exists. In a subclass that conditionally creates it, this is a latent `AttributeError` that fires hours into a training run, not at startup.

MEDIUM BUG-04 — `base.py:128, 339`

`time.strftime("%Y-%m-%d_%H:%M:%S")` produces colons in directory names. Colons are illegal in Windows file paths. `os.makedirs()` crashes on Windows. Fix: use `%H-%M-%S`.

LOW BUG-05 — `attacks/__init__.py:1`

`from ast import Import` imports Python's AST node class for import statements. Unrelated to this package, never used. Remove the line.

LOW BUG-06 — `ShrinkPad.py:91, 152-153`

Commented-out `# breakpoint()` debug artifact. Open `# TODO: DDP training` never filed as an issue.

Q11 — How does context window size and content affect accuracy?

The same question — "How does the training loop work?" — was asked under two conditions:

Dimension	Broad (11 files)	Narrow (1 file)
Accuracy	High	High within scope
Completeness	Full end-to-end trace	Incomplete on dataset construction
Cross-file hallucination risk	Low	Low in scope; high if pushed beyond
Quality of stated limitations	Moderate	Better — correctly said "dataset construction is in the subclass"

Result: Narrow scope with explicit caveats was more reliable. The narrow-context response correctly admitted what it didn't know. The broad-context response risked overconfidence on details from files read earlier with less attention.

Key findings:

- **Context window is not a searchable index.** The model attends over a flat token sequence. Functions in file 8 of 11 receive less attention than those near the start.
- **Absent files are replaced by training data.** When a function isn't in context, Claude substitutes a plausible one from domain knowledge. The substitution looks identical to a correct answer.
- **Load minimum context per question.** Loading everything creates noise, not signal.
- **For existence checks, use file system tools.** Claude's recall is probabilistic; Grep is authoritative.
- **Grounding beats recency.** A file loaded 10 messages ago is less reliable than one just read. Re-read before citing specific line numbers.

Q12 — Documentation and visualization artifacts produced.

Artifact	Path	Description
AI assistant instructions	CLAUDE.md	Environment, architecture, schedule dict contract, 6 bugs with line refs and fixes
Architecture flowchart	docs/diagrams/architecture.md	Mermaid: all system layers
Sequence diagram	docs/diagrams/sequence.md	Mermaid: BadNets training with file:line annotations
Class hierarchy + ER	docs/diagrams/class_diagram.md	Mermaid: inheritance tree + data model
Interactive visualization	docs/visualization/module_map.html	D3.js: 40-node force-directed graph, no server required
Full written report	docs/REPORT.md	Complete analysis with code traces and line references
Bug audit	docs/bug-audit.md	6 bugs with severity, evidence, verification status, and fixes
Prompt experiments	docs/prompt-experiments.md	4 vague vs. structured prompt pairs with analysis
Exploration log	docs/exploration-log.md	Timestamped session log with key prompts and findings
Root report	REPORT.md	12-question formatted answers
Lesson curriculum	Lessons/00_Index.md	11-lesson educational curriculum for BackdoorBox
Lesson 00	Lessons/Lesson00_System_Overview.md	System overview: architecture, 4 exercises, interview prep
Lesson 01	Lessons/Lesson01_Training_Engine.md	base.py walkthrough: schedule dict, bugs, 4 exercises

CLAUDE.md highlights: Complete environment setup, all required and optional schedule dict keys, 6 verified bugs with exact line references and one-line fixes, data requirements, and AI assistant rules (never fabricate benchmark numbers; treat `tests/` as documentation, not unit tests).

Visualization highlights: `module_map.html` is fully self-contained — no npm, no server, no build step. 40 nodes, 3 edge types, hover descriptions, drag/zoom/pan. Open in any browser.

Michael Williams — Mini Project Submission — May 25, 2026

Repository: <https://github.com/mwill20/CodebaseDetective>